

Problema C

O algoritmo Quicksort depende fundamentalmente de uma operação chamada **partição**. O objetivo desta operação é escolher um elemento do vetor, chamado de **pivô**, e reorganizar o vetor de forma que:

1. Todos os elementos menores ou iguais ao pivô fiquem à sua esquerda.
2. Todos os elementos maiores que o pivô fiquem à sua direita.
3. O pivô termine em sua posição correta e definitiva.

Neste problema, o esqueleto do algoritmo recursivo já foi fornecido. Sua tarefa é implementar apenas a função de partição.

```
int particao(int vetor[], int inicio, int fim);
```

O código base para integração é:

```
#include <stdio.h>
```

```
// Função auxiliar para trocar dois elementos de lugar
```

```
void troca(int* a, int* b) {  
    int t = *a;  
    *a = *b;  
    *b = t;  
}
```

```
// Implemente a lógica do particionamento aqui
```

```
// Esta função deve retornar o índice final onde o pivô foi posicionado
```

```
int particao(int vetor[], int inicio, int fim) {  
    // 1. Escolha o pivô (sugestão: use o último elemento vetor[fim])  
    // 2. Percorra o vetor trocando elementos para organizar menores à  
    esquerda  
    // 3. Coloque o pivô na posição correta entre os menores e maiores  
    // 4. Retorne o índice do pivô  
}
```

```
void quickSort(int vetor[], int inicio, int fim) {  
    if (inicio < fim) {  
        int p = particao(vetor, inicio, fim);  
        quickSort(vetor, inicio, p - 1);  
        quickSort(vetor, p + 1, fim);  
    }  
}
```

```

void imprimeVetor(int vetor[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d", vetor[i]);
        if (i < n - 1) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    int n;
    scanf("%d", &n);

    int vetor[n];
    for(int i = 0; i < n; i++){
        scanf("%d", &vetor[i]);
    }

    quickSort(vetor, 0, n - 1);
    imprimeVetor(vetor, n);

    return 0;
}

```

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 1000$), representando a quantidade de números a serem ordenados. A segunda linha contém N números inteiros separados por espaço (podem ser positivos ou negativos).

Saída

Uma única linha contendo os N números ordenados de forma crescente, separados por um espaço. Deve haver uma quebra de linha (\n) ao final da saída.

Exemplo de entrada	Exemplo de saída
5 4 2 8 1 3	1 2 3 4 8